Project Title

Resume-Job Matching & Talent-Marketplace Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
N.Divya sree - 2520090031
S.Vennela - 2520080018

Team Number: 10

Section: 12

Under the guidance of

Ms.Chandusha Kanda



<<Assistant professor, CS&IT>>

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



Declaration

The Mini Project Report entitled **"Resume-Job Matching & Talent-Marketplace Engine"** is a record of Bonafide work of **N. DivyaSree – 2520090031, S.Vennela - 2520080018** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

N. DivyaSree – 2520090031

S.Vennela- 2520080018

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Certificate

This is to certify that the mini project based report entitled "**Resume-Job Matching & Talent-Marketplace Engine"** is a bonafide work done and submitted by **N.Divya sree, S.Vennela - 2520080018** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

Acknowledgement

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide,<<**Ms.Chandusha kanda**>>,<<**Assistant professor**>>, <<**CS&IT**>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

N.Divys sree – 2520090031

S.Vennela - 2520080018

Table of Contents

S.No.	Contents	Page no.
01	Abstract	6
02	Introduction	7
03	Literature Survey	8
04	Problem Statement and Objectives	9
05	Hardware and Software Requirements	10
06	Implementation	11
07	Results & Discussion	12
08	Conclusion & Future Scope	13
09	References	14

1.Abstract

The **Resume-Job Matching & Talent-Marketplace Engine** is a smart recruitment system designed to connect job seekers with suitable job opportunities by analyzing their resumes and comparing their qualifications with job requirements. The main objective of this project is to simplify the recruitment process, reduce manual effort, and improve the accuracy of candidate selection. The system organizes candidate profiles, job postings, skills, and experience details to support efficient searching, filtering, and matching.

The project utilizes various **Data Structures and Algorithms (DSA)** to manage and process recruitment data efficiently. Binary Search Trees (BSTs) are used to organize candidate records, while AVL trees maintain balanced job listings for efficient searching and insertion. B-Trees support skill indexing, and B+ Trees enable efficient retrieval of candidates based on experience ranges. Graph algorithms, including Breadth-First Search (BFS) and Depth-First Search (DFS), help represent relationships between candidates, skills, and job opportunities. Sorting and weighted scoring techniques can be used to rank candidates according to their skills, qualifications, and experience.

The expected outcome is a user-friendly talent marketplace that provides relevant job recommendations, improves candidate-job compatibility, and helps recruiters identify suitable applicants quickly. By integrating efficient data management techniques with intelligent matching logic, the system aims to enhance recruitment productivity, reduce processing time, and create a more organized and effective hiring experience for both candidates and employers.

Team Name:

Tam Logo:



Team Members:

1. [N.Divya sree, 2520090031]
2. [S.Vennela , 2520080018]

2.Introduction

In today's competitive job market, finding suitable candidates for job openings and identifying relevant employment opportunities have become challenging tasks. Traditional recruitment methods often require significant manual effort to review resumes, compare qualifications, and shortlist candidates. These methods can be time-consuming and may result in suitable candidates being overlooked. Therefore, an efficient and organized system is needed to connect job seekers with employers based on their skills, qualifications, and experience.

The **Resume-Job Matching & Talent-Marketplace Engine** is designed to simplify and improve the recruitment process by organizing candidate profiles and job listings and identifying suitable matches. The system uses Data Structures and Algorithms (DSA) to store, search, retrieve, and analyze recruitment-related information efficiently. It incorporates Binary Search Trees (BSTs) for candidate records, AVL trees for balanced job listing management, B-Trees for skill indexing, and B+ Trees for experience-based queries. Graph traversal algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) can represent and explore relationships between candidates, skills, and job opportunities.

The primary objective of this project is to reduce recruitment time, improve matching efficiency, and provide a structured talent marketplace for both job seekers and recruiters. By combining efficient data organization with candidate-ranking techniques, the system aims to make the hiring process faster, more accessible, and more effective.

This report presents the project's objectives, requirements, system design, algorithms, implementation methodology, expected results, and future enhancements.

3.Literature Survey

01	**Resume Matcher (open-source) [1].** An open-source tool that parses a résumé and a job description, extracts keywords and compares them using vector similarity to show how well the résumé fits the job. It helps applicants tailor their résumés and mimics the screening done by Applicant Tracking Systems. It works from the candidate's side only and does not decide which candidate should get which job when many compete.
02	**pyresparser [2].** A Python résumé parser that extracts the name, e-mail, mobile number, skills, experience, education and designation from PDF and DOCX résumés using NLP libraries. It turns unstructured résumés into structured data, which is the first step of any matching system. It does not rank candidates or optimise assignments, which are the focus of this project.
03	**Pattern matching with KMP [3], [9].** The Knuth–Morris–Pratt algorithm searches a pattern in a text in O(n + m) time by never re-examining matched characters, using a prefix (failure) table. It is widely used for keyword spotting in large documents. In this project it is used to locate skill keywords inside résumé and job-description text.
04	**Edit distance for fuzzy matching [4].** Levenshtein distance counts the minimum insertions, deletions and substitutions needed to change one string into another and is computed with Dynamic Programming. It is the standard technique for spelling correction and approximate matching. Here it maps mistyped or variant skill names to a canonical skill vocabulary.
05	**Assignment problem and the Hungarian method [5], [6].** Kuhn's Hungarian method solves the problem of assigning n workers to n tasks at minimum total cost in polynomial time. It is the classical algorithm for optimal one-to-one matching in a weighted bipartite graph. It is applied here to assign candidates to open roles so that the overall match score is maximised.
06	**Stable matching in talent marketplaces [7].** Gale and Shapley's stable-marriage algorithm is the basis of real two-sided markets such as hospital–resident and college admission matching. It shows that matching in a marketplace must consider both sides' preferences, not just a one-sided keyword score. This motivates the two-sided view (jobs for candidates and candidates for jobs) used in this system.
07	**Team formation as set cover [8], [9].** Choosing the fewest people whose combined skills cover all the skills required by a project is an instance of the set-cover problem, which is NP-hard in general. For the small skill sets found in projects, an exact bitmask Dynamic Programming solution is practical. The proposed system uses this idea for its minimum-team staffing analysis.

4.Problem Statement and Objectives

PROBLEM STATEMENT

Manual résumé screening is slow, inconsistent and cannot optimally match many candidates to limited job openings.

PROBLEM DESCRIPTION

Recruiters must compare unstructured résumés with job descriptions by hand, and the same skill may be written in many ways. The system must extract and normalize skills, score every candidate–job pair, rank the results and assign candidates to roles with the best overall fit.

Objectives

The purpose of the project is to reduce the manual effort and error in recruitment by applying Data Structures and Algorithms to résumé–job matching. The objectives are:

01	To extract important skills from résumés and job descriptions using the KMP string-matching algorithm.
02	To normalize skill names (aliases and spelling errors) using edit-distance Dynamic Programming.
03	To compute a similarity score for every candidate–job pair based on skills, qualification and experience.
04	To produce personalised job recommendations for candidates and ranked candidate lists for recruiters using a heap.
05	To assign candidates to roles optimally using the Hungarian algorithm when candidates compete for limited openings.
06	To find the minimum set of people whose skills cover all skills required by a project (team staffing).

5.Hardware and Software Requirements

HARDWARE REQUIREMENTS	SOFTWARE REQUIREMENTS
Processor: Intel Core i3 / AMD Ryzen 3 or higher RAM: 4 GB or more Storage: 20 GB Monitor Internet connection	Programming language: Java JDK 17 or later Editor / IDE: Visual Studio Code Java Collections Framework (HashMap, TreeSet, PriorityQueue, ArrayList) Algorithm implementations: String Matching (KMP) and Dynamic Programming

6.Implementation

6.1 System Architecture

The system is organised in four layers. The input layer receives résumés, job descriptions, skills and qualifications. The processing layer cleans the text, extracts and normalizes skills and calculates similarity. The algorithm engine applies string, Dynamic Programming, graph and optimisation algorithms. The output layer delivers candidate rankings, job recommendations, optimal assignments and staffing analysis.

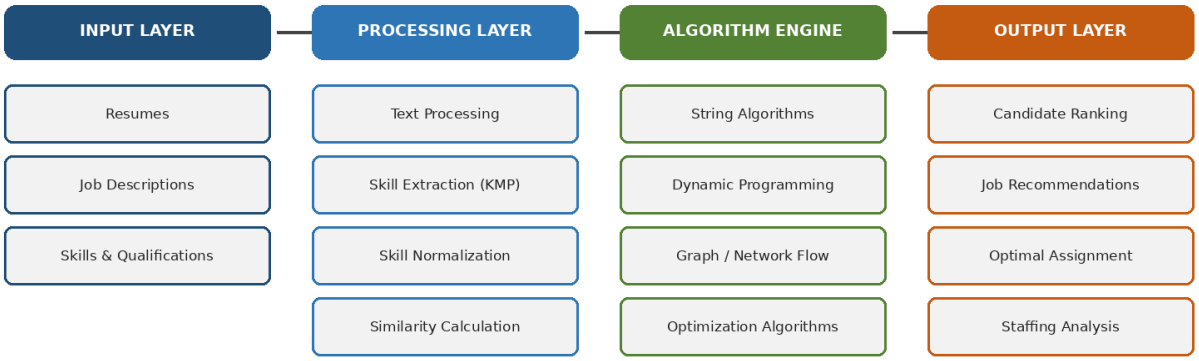


Figure 1: Layered architecture of the Talent-Marketplace Engine

6.2 Methodology

Each résumé flows through the pipeline shown below. Skills are extracted and normalized, every candidate is compared with every job, results are ranked, and finally candidates are assigned to roles.



7.Results & Discussion

The program was run on a sample marketplace of six candidates and five jobs (java TalentMarketplace.java). The output of each stage is shown below.

```
$ java TalentMarketplace.java

=== STEP 1: SKILL EXTRACTION & NORMALIZATION ===
Aarav Mehta  -> [aws, docker, git, java, rest api, spring boot, sql]
Sneha Reddy  -> [machine learning, python, statistics, tensorflow]
Rohan Kumar  -> [css, git, html, javascript, mongodb, node.js, react]
Priya Nair   -> [algorithms, data structures, java, sql]
Kiran Rao    -> [aws, docker, git, kubernetes, python]
Meera Iyer   -> [node.js, python, rest api, sql, statistics]
```

What it shows: KMP extracts skills from the free-text summary and edit distance corrects the skills line. Variants such as “Pyhton”, “ML”, “K8s”, “springboot” and “Amazon Web Services” are mapped to canonical skills.

TalentMarketplace.java). The output of each stage is shown below.

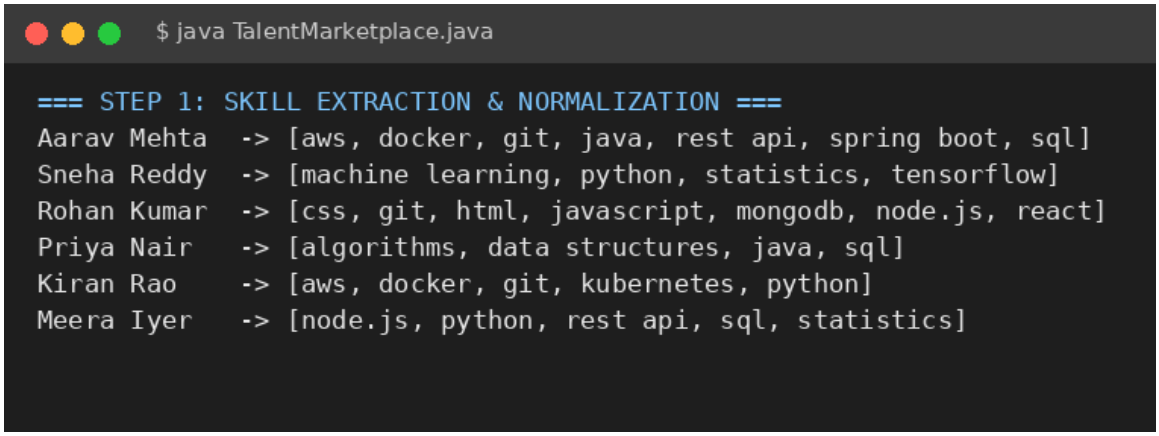


Figure 3: Skill extraction and normalization

What it shows: KMP extracts skills from the free-text summary and edit distance corrects the skills line. Variants such as “Pyhton”, “ML”, “K8s”, “springboot” and “Amazon Web Services” are mapped to canonical skills.

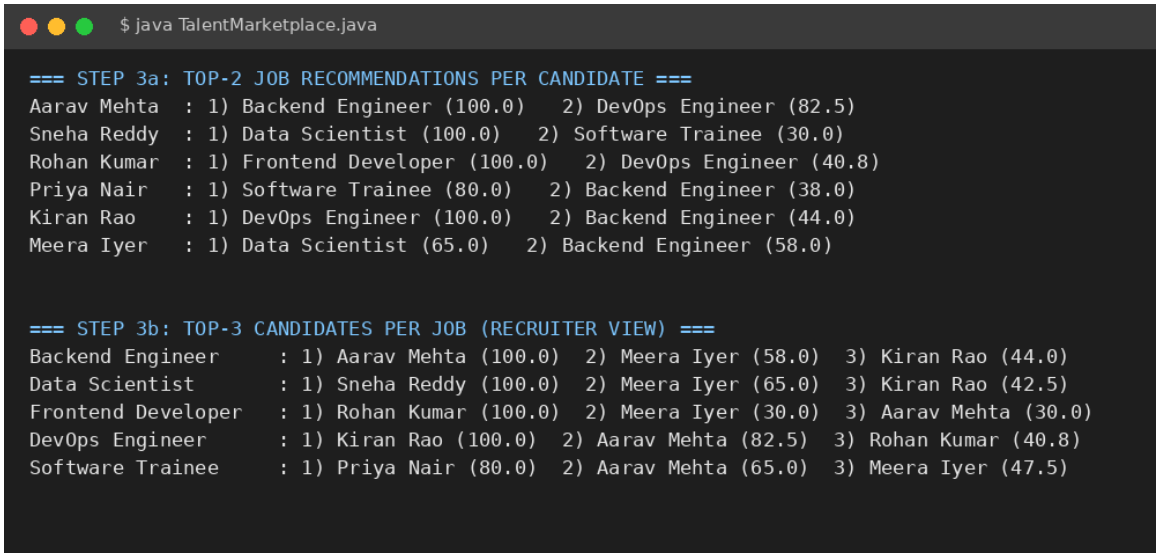
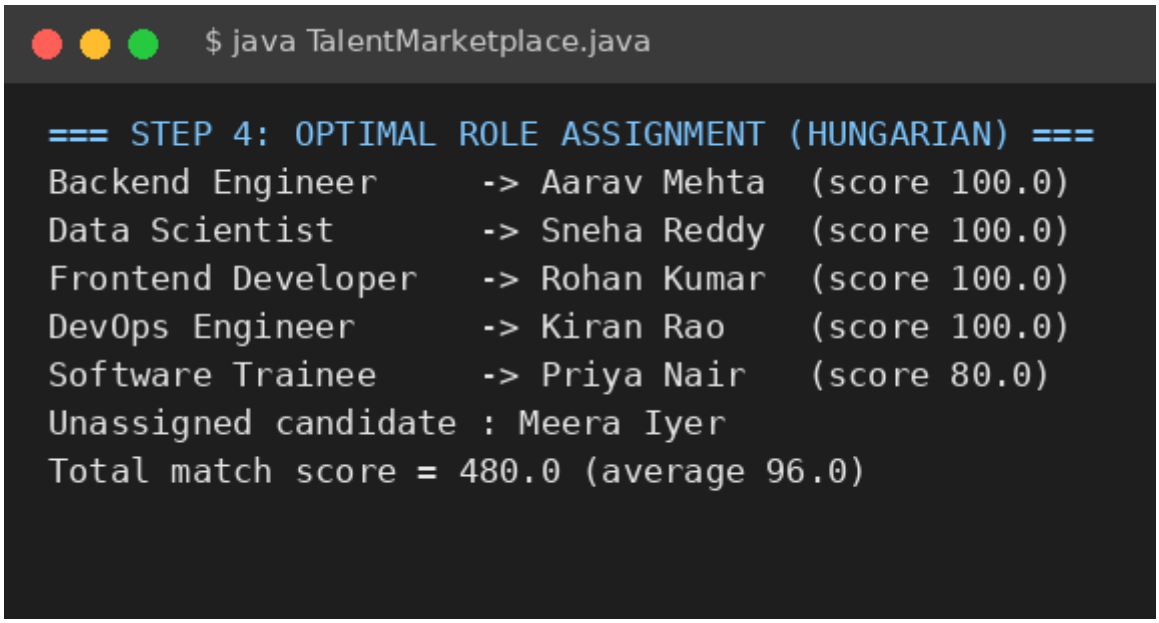


Figure 5: Job recommendations and recruiter rankings

What it shows: A min-heap returns the top-2 jobs for each candidate and the top-3 candidates for each job. This gives candidates personalised recommendations and recruiters a ready shortlist.



8.Conclusion & Future Scope

Conclusion

The project achieved its objectives. Skills were extracted from résumés using KMP and normalized using edit-distance Dynamic Programming; every candidate–job pair received a similarity score; heap-based ranking produced job recommendations for candidates and shortlists for recruiters; the Hungarian algorithm produced an optimal candidate-to-role assignment with an average score of 96.0; and bitmask Dynamic Programming found a minimum team covering all required skills. The work shows how classical Data Structures and Algorithms can reduce manual effort and improve the speed, accuracy and fairness of recruitment decisions.

Future Scope

NEAR TERM	MEDIUM TERM	LONG TERM
Read résumés directly from PDF/DOCX files, extract experience and education automatically, and add a larger skill vocabulary with weights for mandatory and optional skills.	Build a web or desktop interface with a database for candidates and jobs, add two-sided preferences using stable matching, and use semantic similarity (word embeddings) alongside keyword matching.	Deploy as a scalable talent-marketplace service with real-time updates, bias and fairness auditing, and learning from recruiter feedback to improve scoring.

9.References

[1] S. Bhatia (srbhr), “Resume Matcher,” GitHub repository. https://github.com/srbhr/Resume-Matcher

[2] O. Pathak, “pyresparser: A simple resume parser used for extracting information from resumes,” GitHub repository. https://github.com/OmkarPathak/pyresparser

[3] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977. https://doi.org/10.1137/0206024

[4] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals,” Soviet Physics Doklady, vol. 10, no. 8, 1966. https://en.wikipedia.org/wiki/Levenshtein_distance

[5] H. W. Kuhn, “The Hungarian method for the assignment problem,” Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955. https://doi.org/10.1002/nav.3800020109

[6] “Hungarian algorithm,” Wikipedia. https://en.wikipedia.org/wiki/Hungarian_algorithm

[7] D. Gale and L. S. Shapley, “College Admissions and the Stability of Marriage,” American Mathematical Monthly, vol. 69, no. 1, 1962. https://en.wikipedia.org/wiki/Stable_marriage_problem

[8] “Set cover problem,” Wikipedia. https://en.wikipedia.org/wiki/Set_cover_problem

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[10] Oracle, “Java Collections Framework Overview.” https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html